

CSCE 640 QUANTUM ALGORITHMS · MODULE 07 · LECTURE NOTES

The Quantum Fourier Transform

The one piece of machinery that buys an exponential advantage.

MODULE OUTCOMES

Define the QFT and state what it does to a periodic state.

Derive its efficient circuit.

Explain why the exponential-size transform is cheap.

Explain what the QFT does not give you.

Implement and verify it.

1 The transform

$$\text{QFT: } |x\rangle \rightarrow (1/\sqrt{N}) \sum_y \omega^{xy} |y\rangle, \text{ with } \omega = e^{2\pi i/N}, N = 2^n$$

- It is the ordinary discrete Fourier transform, applied to the vector of amplitudes rather than to a list of data values — which is the distinction everything in §1's callout turns on.
- On a state whose amplitudes are periodic with period r , the transform concentrates amplitude near $y \approx k \cdot N/r$ — because the terms at other y have phases spread roughly uniformly around the unit circle and sum to nearly zero — which is Module 04's cancellation in its single most useful instance.
- So measuring y gives an estimate of some random multiple of N/r — one multiple per run — from which r is recovered by continued fractions (Module 08 §3), exactly as Simon's algorithm recovered s from random linear constraints (Module 05 §3).
- **Period becomes peak: that is the sentence to remember**, and it is the whole reason this transform appears in the one algorithm everybody has heard of.

AND THE CLASSICAL COMPARISON IS THE THING TO BE CAREFUL ABOUT

The QFT acts on a vector of 2^n amplitudes using only $O(n^2)$ gates — which looks very much like an exponential speedup over the classical FFT's $O(N \log N)$ on a vector of the same length.

And it is not one, in any usable sense: you cannot read out the transformed vector — one measurement gives you one sample drawn from it (Module 02 §2's bottleneck), and reconstructing the vector takes exponentially many samples.

So the QFT is not a faster Fourier transform — it is a device for making a period detectable in a single sample — which is a much narrower and genuinely useful thing, and is all that Shor's algorithm needs.

Which is the most important caveat in the course, because 'quantum computers do Fourier transforms exponentially faster' is a claim people build application proposals on, and it does not mean what it appears to mean (Module 11 §4 and Module 13 §3 both return to this).

2 The circuit

THE KEY FACTORISATION

the QFT's output amplitude factorises across the output qubits -- each output bit's state depends on the input through a product of phases. So the transform is a PRODUCT of single-qubit operations with phase corrections, not a general 2^n by 2^n matrix.

THE CIRCUIT, for qubit j from most significant

- 1 H on qubit j
- 2 controlled- R_k from each less significant qubit, $k = 2, 3, \dots$, with phase $2\pi / 2^k$
- 3 move to the next qubit
- 4 reverse the qubit order at the end

GATE COUNT

n Hadamards + $n(n-1)/2$ controlled rotations
 $= O(n^2)$

AND THE SMALL ROTATIONS CAN BE DROPPED

phases below the accuracy you need contribute negligibly: the approximate QFT uses $O(n \log n)$ gates, which matters a lot (M10)

The approximate QFT drops the tiny rotations and uses $O(n \log n)$ gates — which is what makes Shor's circuit depth tolerable rather than merely polynomial, and is a good example of an approximation that costs nothing because the discarded terms are below the measurement's resolution anyway. The final bit-reversal is the step most often omitted in an implementation, and it is where ordering bugs live (§4).

BECAUSE THE TRANSFORM FACTORISES, WHICH IS ALSO WHY THE CLASSICAL FFT IS FAST

The classical FFT is fast for the same structural reason — the DFT matrix factors into a product of sparse matrices — and the quantum circuit is that same factorisation realised as a sequence of gates rather than as a recursion.

Which is reassuring: no new mathematical magic is involved, only the fact that a unitary with this particular structure happens to have a short circuit — and the structure was known long before quantum computing.

And it is a general lesson worth extracting: a unitary has an efficient circuit when it factorises, which is rare (Module 03 §1's counting argument: there are vastly more unitaries than short circuits) **and is exactly why useful quantum algorithms are scarce** rather than abundant.

So the QFT is precious precisely because it is both cheap and useful — most unitaries are neither — and this single piece of machinery underwrites the subject's one dramatic result, which is a lot of weight for one circuit to carry.

3 What it gives

- **Period finding, which is phase estimation, which is Shor's algorithm — the one chain of reasoning in this subject that leads to a superpolynomial advantage** (Module 08), and it is worth seeing as a single chain rather than three topics.
- **The hidden subgroup problem over abelian groups** — which generalises period finding neatly, **and whose non-abelian cases are open and would matter a great deal** if solved: graph isomorphism and certain lattice problems are non-abelian HSP instances, and lattice problems are what post-quantum cryptography rests on (CSCE 711 Module 13).
- **And that is approximately the list — which is worth stating plainly, because the QFT's prominence in every introduction suggests a far wider applicability than it actually has.**
- **It does not give you signal processing**, because **you cannot read out the spectrum** (§1's callout) — so 'quantum FFT for audio or images' is not a thing, however often it is proposed.
- **And it does not give you data analysis**, for that reason plus the input problem: **loading a classical dataset into amplitudes is itself expensive** — generally requiring time comparable to the data size, which destroys any polynomial advantage before the algorithm starts (Module 11 §4). **The input problem undermines most proposed data-analysis applications**, and is the first question to ask of one.

4 Implementing it

VERIFY THE QFT ON A KNOWN PERIOD BEFORE BUILDING ANYTHING ON IT

Prepare a state whose amplitudes are periodic with a known r , apply your QFT, and check that the outcome distribution peaks at multiples of N/r — which is a complete functional test and takes a few minutes.

And check the inverse: QFT followed by inverse QFT must be the identity — which catches qubit-ordering errors — **and the final bit-reversal step is where those errors live**, in essentially every first implementation.

Then compare directly against the classical FFT of your amplitude vector — your simulator has the whole vector, so you can check the transform exactly, amplitude by amplitude, which real hardware cannot and which no amount of sampling would establish.

Which is the practical value of having written Module 02 §4's simulator yourself: you can inspect the amplitudes that the model says are unobservable — and that is how to debug a quantum algorithm before trusting it, which is the only debugging method available.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nielsen & Chuang, chapter 5 (library copy)	§§1 and 2 — the QFT, its circuit, and phase estimation, in the standard development.
Coppersmith — An approximate Fourier transform useful in quantum factoring (free)	§2's approximate version in the original — and the argument for dropping the small rotations.
Kitaev, Shen & Vyalıi, the phase estimation chapter	§1 and Module 08 — the cleanest derivation, and the one that makes the eigenvalue framing central. Library copy.
Aaronson — Read the fine print (free)	§3's input problem and non-uses — the standard reference for what the caveats actually cost, and it should be read before any quantum machine learning proposal.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write the QFT matrix** for $N = 4$ and verify it is unitary.
- 2 **Apply it by hand** to a state with period 2.
- 3 **State why it is not a faster FFT**, in three sentences.
- 4 **Derive the factorisation** that makes the circuit short.
- 5 **Implement the QFT circuit** and compare to the matrix.
- 6 **Verify QFT then inverse QFT is the identity**, and find your ordering bug.
- 7 **Implement the approximate QFT** and measure the fidelity loss.
- 8 **Count gates for both** at $n = 10, 20, 50$.
- 9 **Test your QFT on three known periods** and check the peaks.
- 10 **Estimate the cost of loading a classical vector** into amplitudes, and state the consequence.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define the QFT and say what it does to a periodic state.
- 2 What is the one sentence to remember?
- 3 Why is it not a faster Fourier transform?

- 4 Why is that the course's most important caveat?
- 5 Why does $O(n^2)$ suffice?
- 6 What does the approximate QFT drop, and what does it cost?
- 7 When does a unitary have an efficient circuit, and why is that rare?
- 8 List the QFT's actual uses.
- 9 Why is signal processing not among them?
- 10 State the input problem and its consequence.